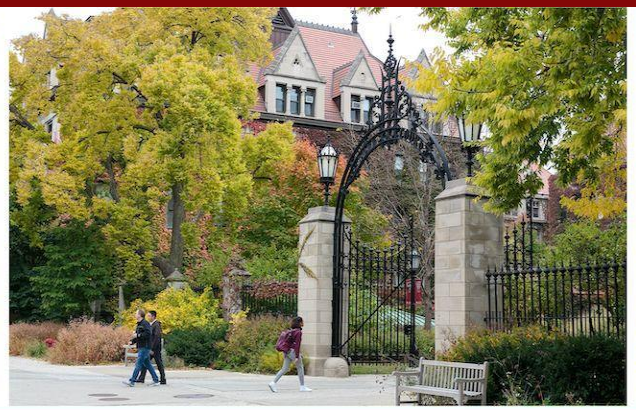
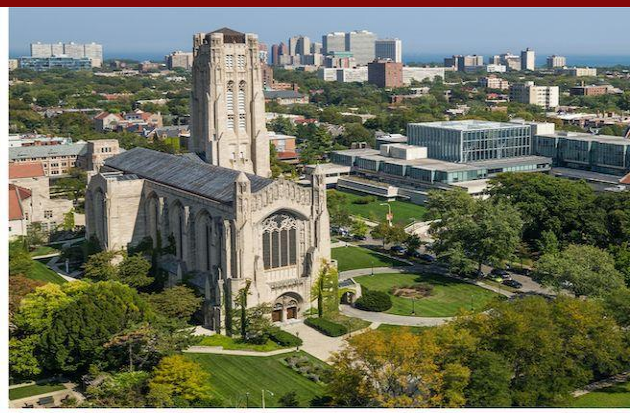
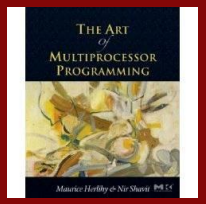
MPCS 52060 - Parallel Programming
M3: Principles of Mutual Exclusion



Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

M3 Objective: Locks

- How we are going to ensure determinism when multiple threads want to enter in the critical section?

Critical sections, locks: Basic idea

- How do we ensure only one thread at a time uses a given shared resource?
- If one thread uses the resource, it needs to lock out all other threads.
- The idea was used, at least, since 1849...

A real-world analogy



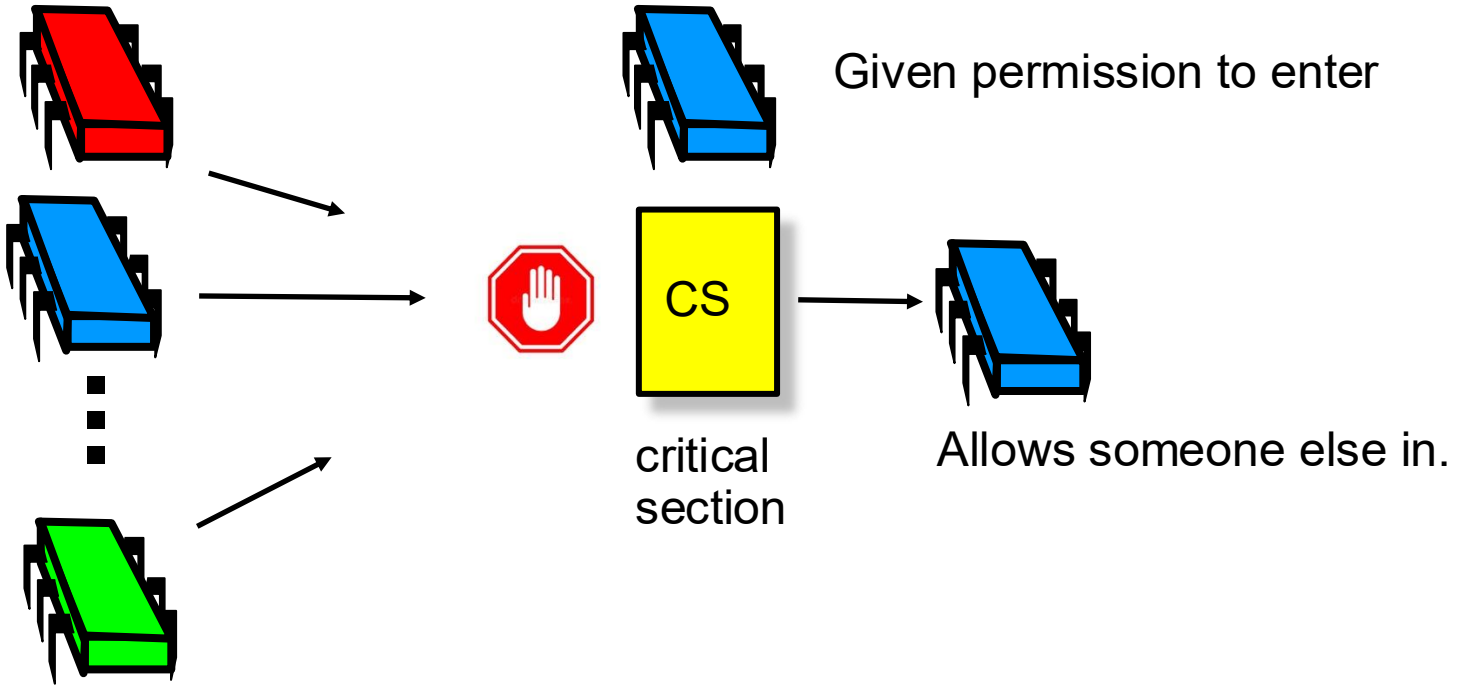
Critical sections, locks: Basic idea



https://upload.wikimedia.org/wikipedia/commons/3/30/TokenRingSanta.jpg

M3 Objective

- How we are going to ensure determinism when threads want to enter in the critical section?



Formalizing Concurrent Computation

Two types of formal properties in asynchronous computation:

- Safety Properties:
  - Nothing bad happens ever
  - For example - a traffic light never displays green in all directions, even if power fails.
- Liveness Properties:
  - States that a particular “good” thing will happen.
  - For example - a red light will eventually turn green.

Formalizing Critical Sections

- Synchronization primitives need to adhere to the following properties and principles about critical sections in order to be correct:
  - Mutual Exclusion Property:
    - Critical sections of different threads do not overlap. Only one thread is executing a critical section at a time
    - Guarantees that a computation's results are correct.
    - This is a safety property.
  - Deadlock-freedom property:
    - If multiple threads simultaneously request to enter a critical section, then it must allow one to proceed
    - Threads outside the critical section have no say in which thread can proceed into the critical section, only those currently waiting have influence.
    - It implies the system never "freezes".
    - This is a liveness property.

Formalizing Critical Sections

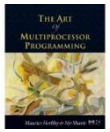
- Starvation-freedom property:
  - Every thread that attempts to acquire the lock eventually succeeds.
  - Many mutual exclusive algorithms in practice are not starvation free because its less likely starvation will occur in those algorithms.
  - There is no guarantee on how long thread will wait to acquire the lock.
  - Also known as lockout freedom or bounded-waiting
  - This is a liveness property.
- Fairness Principle:
  - A thread who just left the critical section cannot immediately re-enter the critical section if other threads have already requested to enter the critical section.
  - Some algorithms place bounds on how long a thread can wait.

Demo/thought experiment

- How can we implement a lock?

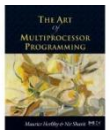
Implementing Locks
Two-Thread vs *n*-Thread Solutions

- 2-thread solutions first
  - Illustrate most basic ideas
  - Fits on one slide
- Then *n*-thread solution
- You will never use these protocols
  - Get over it
- You are advised to understand them
  - The same issues show up everywhere
  - Except hidden and more complex



Two-Thread Conventions

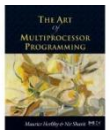
```
class … implements Lock {
  …
  // thread-local index, 0 or 1
  public void lock() {
    int i = ThreadID.get();
    int j = 1 - i;
  …
  }
}
```



Two-Thread Conventions

```
class … implements Lock {
  …
  // thread-local index, 0 or 1
  public void lock() {
    int i = ThreadID.get();
    int j = 1 - i;
  …
  }
}
```

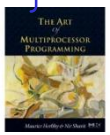
Henceforth: i is current thread, j is other thread



LockOne

```
class LockOne implements Lock {
private boolean[] flag = new boolean[2];
public void lock() {
  int i = ThreadID.get();
  int j = 1 - i;
  flag[i] = true;
  while (flag[j]) {}
 }
public void unlock() {
  int i = ThreadID.get();
  flag[i] = false;

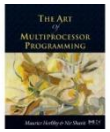
}
```



LockOne

```
class LockOne implements Lock {
private boolean[] flag = new boolean[2];
public void lock() {
  flag[i] = true;
  while (flag[j]) {}
 }
```

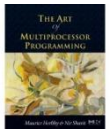
Each thread has flag



LockOne

```
class LockOne implements Lock {
private boolean[] flag = new boolean[2];
public void lock() {
  flag[i] = true;
   while (flag[j]) {}
  }
```

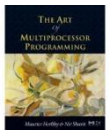
Set my flag



LockOne

```
class LockOne implements Lock {
private boolean[] flag = new boolean[2];
public void lock() {
  flag[i] = true;
  while (flag[j]) {}
  }
```

Wait for other flag to become false

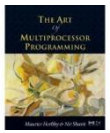


Deadlock Freedom

- LockOne Fails deadlock-freedom
  - Concurrent execution can deadlock

```
flag[i] = true;      flag[j] = true;
while (flag[j]){}    while (flag[i]){}
```

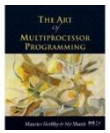
  - Sequential executions OK



LockTwo

```
public class LockTwo implements Lock {
 private int victim;
 public void lock() {
   int i = ThreadID.get();
   int j = 1 - i;
  victim = i;
  while (victim == i) {};
 }

 public void unlock() {}
}
```

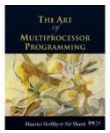


LockTwo

```
public class LockTwo implements Lock {
 private int victim;
 public void lock() {
  victim = i;
  while (victim == i) {};
 }

 public void unlock() {}
}
```

Let other go first

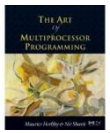


LockTwo

```
public class LockTwo implements Lock {
 private int victim;
 public void lock() {
 victim = i;
  while (victim == i) {};
  }

 public void unlock() {}
}
```

Wait for permission

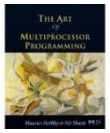


LockTwo

```
public class Lock2 implements Lock {
 private int victim;
 public void lock() {
  victim = i;
  while (victim == i) {};
 }

 public void unlock() {}
}
```

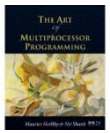
Nothing to do



LockTwo Claims

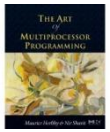
- Satisfies mutual exclusion
  - If thread `i` in critical section
  - Then **`victim == j`**
  - Cannot be both 0 and 1
- Not deadlock free
  - Sequential execution deadlocks
  - Concurrent execution does not

```
public void LockTwo() {
  victim = i;
  while (victim == i) {};
 }
```



Peterson's Algorithm

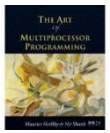
```
public void lock() {
  int i = ThreadID.get();
  int j = 1 - i;
  flag[i] = true;
  victim  = i;
  while (flag[j] && victim == i) {};
}
public void unlock() {
 flag[i] = false;
}
```



Peterson's Algorithm

Announce I'm interested

```
public void lock() {
 flag[i] = true;
 victim  = i;
 while (flag[j] && victim == i) {};
}
public void unlock() {
 flag[i] = false;
}
```

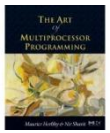


Peterson's Algorithm

Announce I'm interested

Defer to other

```
public void lock() {
 flag[i] = true;
 victim  = i;
 while (flag[j] && victim == i) {};
}
public void unlock() {
 flag[i] = false;
}
```



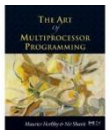
Peterson's Algorithm

```
public void lock() {
  flag[i] = true;
  victim  = i;
  while (flag[j] && victim == i) {};
}
public void unlock() {
  flag[i] = false;
}
```

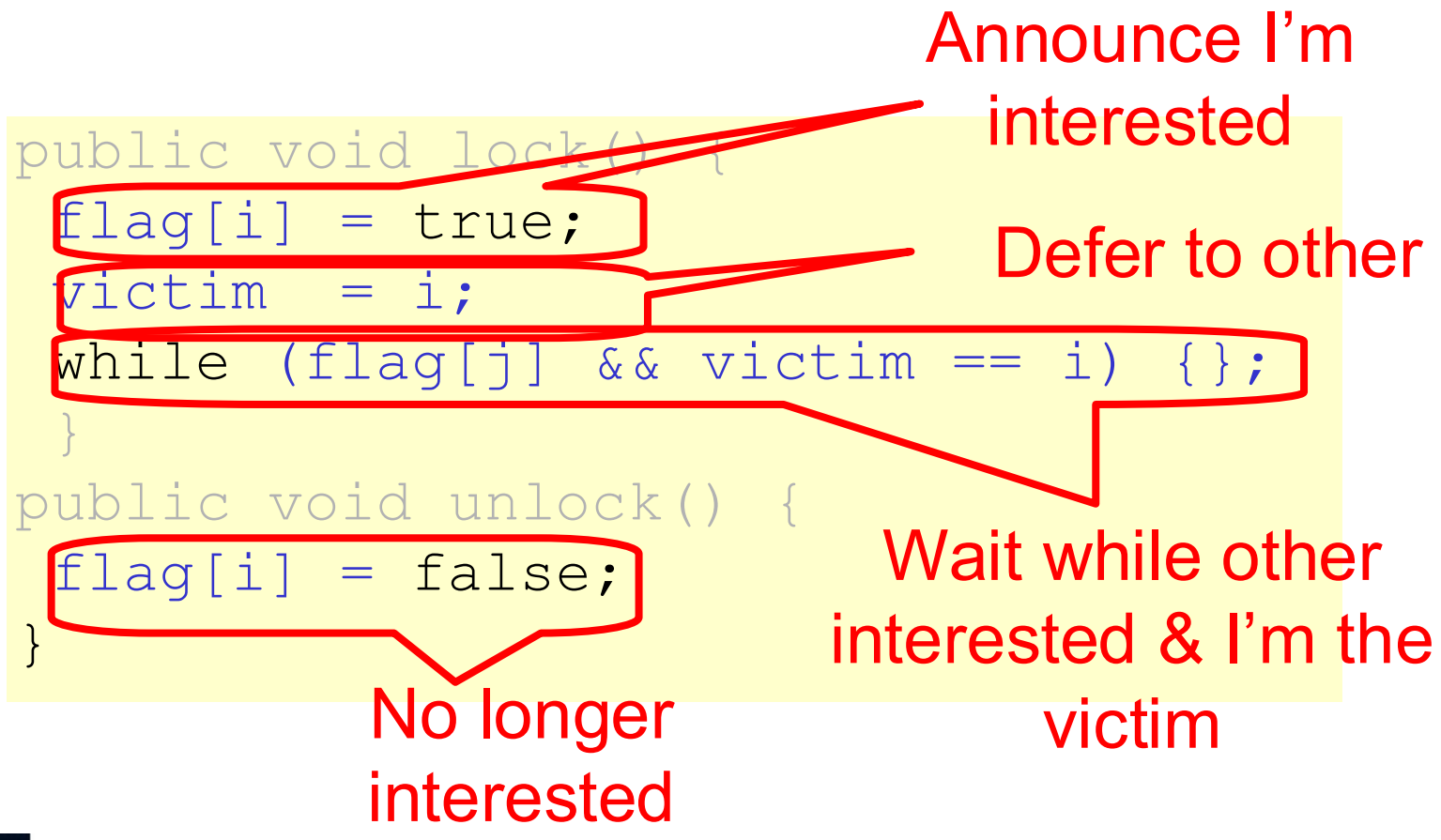
Announce I'm interested

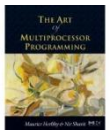
Defer to other

Wait while other interested & I'm the victim



Peterson's Algorithm





Peterson's Algorithm

- Satisfies mutual exclusion & deadlock freedom properties
  - Uses both lock-one and lock-two strategies
- Downside: only works for two threads

